TABLE OF CONTENT

Task 1 : Environmental Setup 2
Task 2 : Dataset Curation 2
a. Loading the dataset 2
b. Checking for Corrupted files 3
c. Splitting the Dataset 3
d. Visualization of random samples 6
Task 3 : Model Building 8
a. Model Development 8
b. Model Performance 9
c. Attribute Regularization 10
d. Model Regularization 11
Task 4 : Results and Discussion 13
a. Summary of experimental results 13
b. Comparision on model results 13
c. Analysis of hyperparameter and other model settings 14
d. Methodology and Implementation Strategies 16
e. Future Direction 17

Task 1 : Environmental Setup

The development environment was Google Colab and Tensorflow with Keras was used to develop the python script.

Below Table I shows the important libraries and their purpose of use within the script.

Table I: Libraries used for the assignment

Library/Functions	**Usage**
from google.colab import drive	To mount the google drive
import numpy as np	To work with image arrays
import tensorflow as tf	Machine Learning platform
from tensorflow import keras	Machine Learning platform
import os	To work with directories; creating, checking availability etc.
import shutil	To work with files, copying, listing, moving etc.
import random	For random shuffling of images
import glob	Searching patterns in file paths and names
import matplotlib.image as mpimg	For reading the images
import matplotlib.pyplot as plt	For plotting graphs
from tensorflow.keras.models import Sequential	To build sequential CNN model
from tensorflow.keras.layers import Activation, Dense, Flatten, Conv2D, MaxPool2D, Dropout, BatchNormalization	For getting layers, functions related to CNN model
from tensorflow.keras.optimizers import Adam	As the optimizer function in CNN
from tensorflow.keras.metrics import categorical_crossentropy	As the loss function of the model
from tensorflow.keras.preprocessing.image import ImageDataGenerator	For creating batches of images with augmentations

Task 2 : Dataset Curation

a. Loading the dataset

First, the zipped dataset was uploaded to the working directory called “Deep Learning” on the drive. Then the drive was mounted to the Colab Notebook and loaded the unzipped dataset into the Notebook. Set a base path to the working directory. The relevant code segments were given below in Figure 1.

```
drive.mount('/content/gdrive')
```

```
[19] base_path =  "/content/gdrive/MyDrive/Deep Learning/"
```

```
# Change the working directory to the base_path
os.chdir(base_path)
```

```
[22] # Unzip the dataset
     if os.path.isdir('dataset2') is False:
       !unzip -q dataset2.zip
```

Figure 1: Mounting Drive, Unzip Dataset, Setting Base_path

b. Checking for Corrupted files

Next, the dataset was checked to find the availability of any non-image files and all the files were opened to verify whether they are corrupted. The following code snippet in figure 2 has been used for this purpose. There were no corrupted files and all of the files were images.

```
[23] # Checking whether all files are images and they can be opened.

     images = []
     for filename in os.listdir('dataset2'):
         try:
             img = mpimg.imread(os.path.join('dataset2', filename))
             if img is not None:
                 images.append(img)
         except:
             print('Cant import ' + filename)
     images = np.asarray(images)
```

Figure 2: Checking for corrupted files

c. Splitting the Dataset

The dataset consists of 1125 images belonging to 4 classes; cloudy, rain, shine and sunrise. The corresponding class name was included in the file name of all the images and that information was used to identify their classes. First, the dataset was divided into 4 directories; cloudy, rain, shine and sunrise by extracting the label on the file name using the following code in figure 3.

```python
# Organize the data into directories

os.chdir('dataset2')

if os.path.isdir('cloudy') is False:
    os.makedirs('cloudy')
    os.makedirs('rain')
    os.makedirs('shine')
    os.makedirs('sunrise')

    for i in glob.glob('cloudy*'):
        shutil.move(i, 'cloudy')
    for i in glob.glob('rain*'):
        shutil.move(i, 'rain')
    for i in glob.glob('shine*'):
        shutil.move(i, 'shine')
    for i in glob.glob('sunrise*'):
        shutil.move(i, 'sunrise')

os.chdir('../')
```

Figure 3: Dividing the dataset according to four labels

Next, all the images within above directories were copied into test and train directories according to 70% train and 30% test split ratio (See figure 4).

```python
rootdir= 'dataset2' #path of the original folder

classes = ['cloudy', 'rain', 'sunrise', 'shine']

#below steps will be performed if there is no previously created train directory
if os.path.isdir('dataset2/train/cloudy') is False:

  for i in classes:
    os.makedirs(rootdir +'/train/' + i)

    os.makedirs(rootdir +'/test/' + i)

    source = rootdir + '/' + i

    allFileNames = os.listdir(source)

    np.random.shuffle(allFileNames)

    test_ratio = 0.3

    train_FileNames, test_FileNames = np.split(np.array(allFileNames), [int(len(allFileNames)* (1 - test_ratio))]

    train_FileNames = [source+'/'+ name for name in train_FileNames.tolist()]
    test_FileNames = [source+'/' + name for name in test_FileNames.tolist()]

    for name in train_FileNames:
      shutil.copy(name, rootdir +'/train/' + i)

    for name in test_FileNames:
      shutil.copy(name, rootdir +'/test/' + i)
```

Figure 4:Figure 4: Splitting the dataset into Train and Test sets

The final directory structure is given below. The Train and Test directories got 786 and 339 images respectively.

- Deep Learning
  - Dataset2
    - Cloudy
    - Rain
    - Shine
    - Sunrise
    - Test
      - Cloudy
      - Rain
      - Shine
      - Sunrise
    - Train
      - Cloudy
      - Rain
      - Shine
      - Sunrise

Then these image directories with class labels were used to generate the batches of images. Here, no data augmentation was used on images. The relevant code segment is given in Figure 5 below.

```
[83] # Data Generator
     datagen = tf.keras.preprocessing.image.ImageDataGenerator(rescale=1./255,
                                                               )

     train_gen = datagen.flow_from_directory(directory = TRAIN_PATH,
                                             batch_size = BATCH_SIZE,
                                             seed = 1,
                                             color_mode = 'rgb',
                                             shuffle = True,
                                             classes=classes,
                                             #class_mode = 'binary',
                                             target_size = (IMAGE_SIZE, IMAGE_SIZE))

     test_gen = datagen.flow_from_directory(directory = TEST_PATH,
                                            batch_size = BATCH_SIZE,
                                            seed = 1,
                                            color_mode = 'rgb',
                                            shuffle = True,
                                            classes=classes,
                                            #class_mode = 'binary',
                                            target_size = (IMAGE_SIZE, IMAGE_SIZE))
```

Figure 5: Generating image batches

d. Visualization of random samples

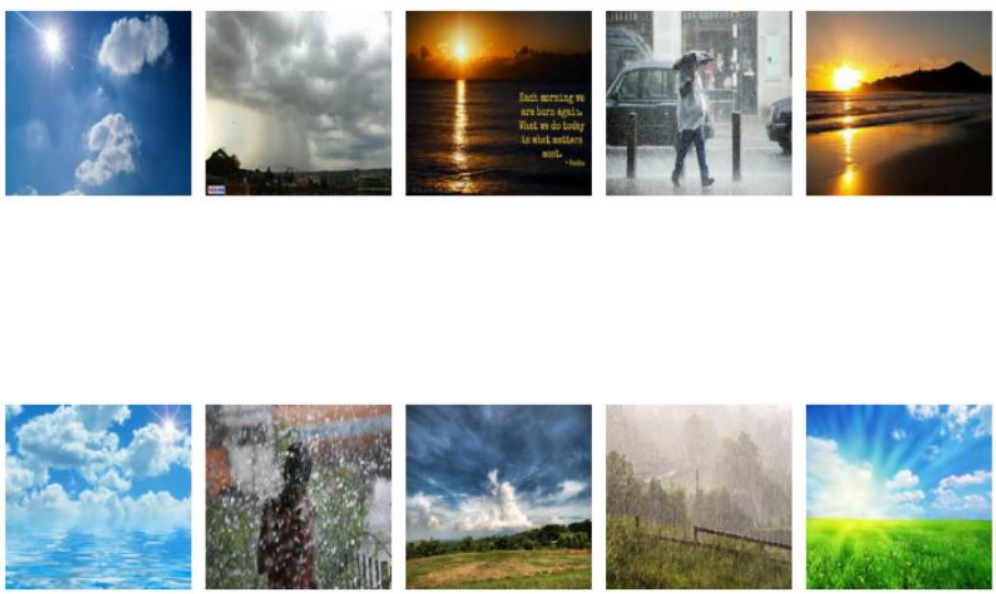
A random sample of 10 images was displayed from both training and testing sets using the following code in Figures 6 and 7 respectively.

```
[188] # Ploting Images - 10 Training samples
      images, labels = next(train_gen)
      fig, axes = plt.subplots(2, 5, figsize = (10, 10))

      axes = axes.flatten()

      for img, ax in zip(images, axes):
          ax.imshow(img.reshape(IMAGE_SIZE, IMAGE_SIZE, 3))
          ax.axis('off')
      plt.tight_layout()
      plt.show()
      print(labels)
```

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
```



```
[[0. 0. 0. 1.]
 [1. 0. 0. 0.]
 [0. 0. 1. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]
 [0. 1. 0. 0.]
 [1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 0. 1.]]
```

Figure 6: Visualizing random images from the training set

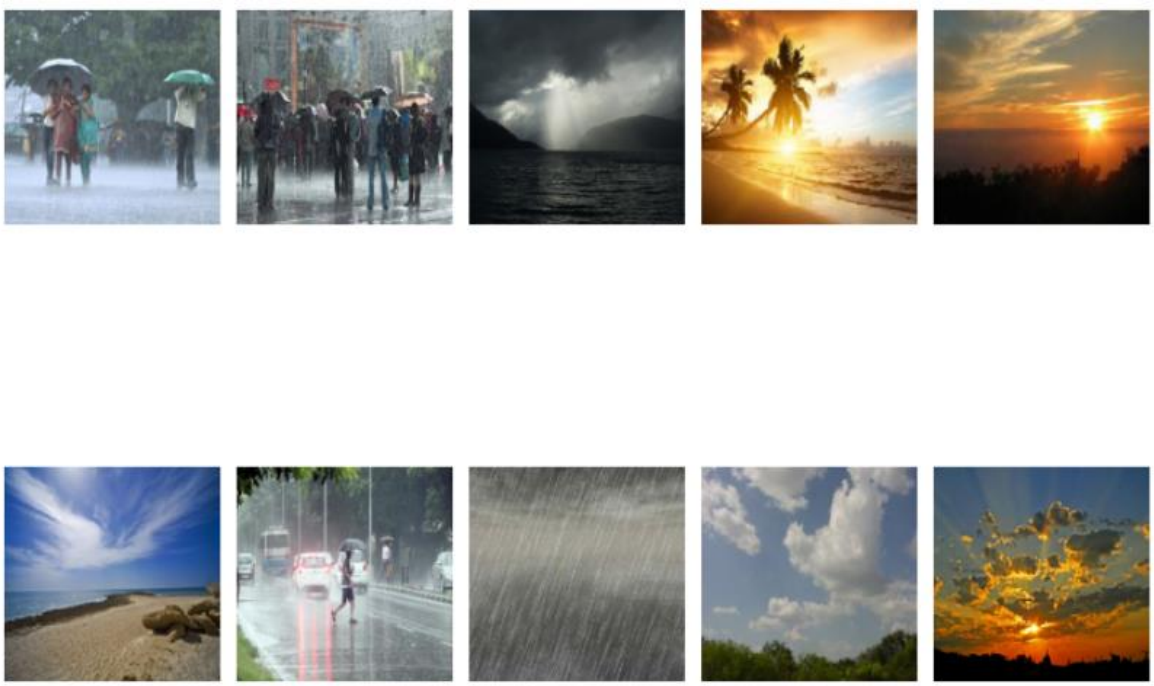
```python
# Ploting Images - 10 Testing samples
images, labels = next(test_gen)
fig, axes = plt.subplots(2, 5, figsize = (10, 10))

axes = axes.flatten()

for img, ax in zip(images, axes):
    ax.imshow(img.reshape(IMAGE_SIZE, IMAGE_SIZE, 3))
    ax.axis('off')

plt.tight_layout()
plt.show()
print(labels)
```

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
```



```
[[0. 1. 0. 0.]
 [0. 1. 0. 0.]
 [1. 0. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]
 [0. 1. 0. 0.]
 [0. 1. 0. 0.]
 [1. 0. 0. 0.]
 [0. 0. 1. 0.]]
```

Figure 7: Visualizing random sample of 10 images from the testing set

Task 3 : Model Building

a. Model Development

A Convolutional Neural Network with 05 convolutional layers was built. The size of the input image was mentioned as (256x256x3) in the first convolutional layer. Each layer contains an increasing number of neurons and a padding was added to preserve the image size. The activation function of each layer is “relu”, except in the last layer which is “softmax”. The relevant code snippet and the summary of the model are given below in figure 8.

```
# Build the CNN model
model = Sequential([
    Conv2D(filters=8, kernel_size=(3, 3), activation='relu', padding = 'same', input_shape=(256,256,3)),
    MaxPool2D(pool_size=(2, 2), strides=2),
    Conv2D(filters=16, kernel_size=(3, 3), activation='relu', padding = 'same'),
    MaxPool2D(pool_size=(2, 2), strides=2),
    Conv2D(filters=32, kernel_size=(3, 3), activation='relu', padding = 'same'),
    MaxPool2D(pool_size=(2, 2), strides=2),
    Conv2D(filters=64, kernel_size=(3, 3), activation='relu', padding = 'same'),
    MaxPool2D(pool_size=(2, 2), strides=2),
    Conv2D(filters=128, kernel_size=(3, 3), activation='relu', padding = 'same'),
    MaxPool2D(pool_size=(2, 2), strides=2),
    Flatten(),
    Dense(units=4, activation='softmax')
])
```

```
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d_5 (Conv2D)           (None, 256, 256, 8)       224

 max_pooling2d_5 (MaxPooling  (None, 128, 128, 8)      0
 2D)

 conv2d_6 (Conv2D)           (None, 128, 128, 16)      1168

 max_pooling2d_6 (MaxPooling  (None, 64, 64, 16)       0
 2D)

 conv2d_7 (Conv2D)           (None, 64, 64, 32)        4640

 max_pooling2d_7 (MaxPooling  (None, 32, 32, 32)       0
 2D)

 conv2d_8 (Conv2D)           (None, 32, 32, 64)        18496

 max_pooling2d_8 (MaxPooling  (None, 16, 16, 64)       0
 2D)

 conv2d_9 (Conv2D)           (None, 16, 16, 128)       73856

 max_pooling2d_9 (MaxPooling  (None, 8, 8, 128)        0
 2D)

 flatten_1 (Flatten)         (None, 8192)              0

 dense_1 (Dense)             (None, 4)                 32772

=================================================================
Total params: 131,156
Trainable params: 131,156
Non-trainable params: 0
```

Figure 8: CNN model

Then as given in figure 9, the build model was compiled and trained for 100 epochs.

```
[39] # Model Compile
     model.compile(optimizer=Adam(learning_rate=0.0001), loss='categorical_crossentropy', metrics=['accuracy'])
```

```
[45] # Model training
     STEP_SIZE_TRAIN = train_gen.n // BATCH_SIZE
     STEP_SIZE_TEST = test_gen.n // BATCH_SIZE

     history = model.fit(
         x=train_gen,
         steps_per_epoch=STEP_SIZE_TRAIN,
         validation_data=test_gen,
         validation_steps=STEP_SIZE_TEST,
         epochs=EPOCHS,
         verbose=2
     )
```

Figure 9: CNN model compile and training

b. Model Performance

The performance of training and testing of the model was plotted using the accuracy and loss functions. According to figure 10, the testing performance was good as the accuracy was increasing and loss was decreasing and settling at a constant level. However, the model testing was not performed well compared to the training where the accuracy is always below than the training accuracy and loss was rapidly increasing.

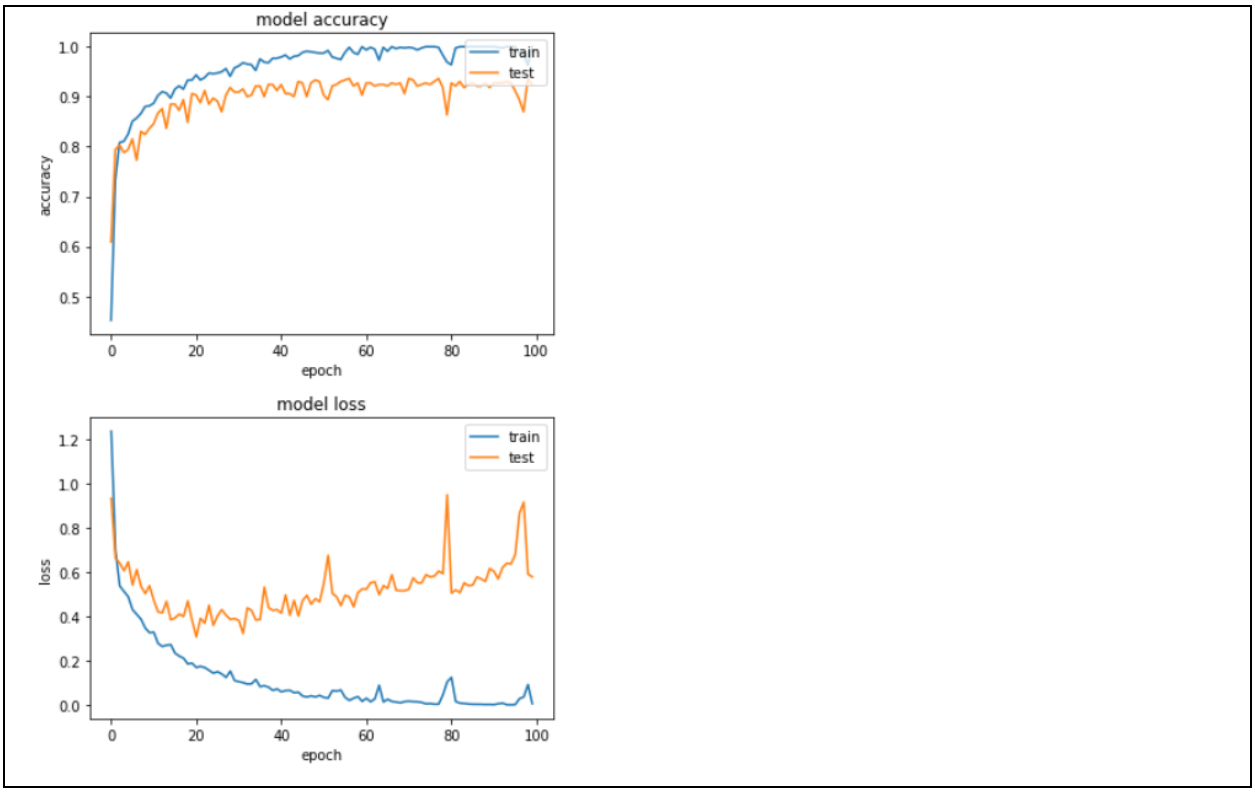


Figure 10: Performance plots of training and testing

c. Attribute Regularization

Later, 05 data augmentation techniques, such as rotating, hieght_shifting, rescale, shearing, zooming, horizontal flipping, were applied to the dataset and re-trained the same model. The relevant code segment is given in figure 11 and the performance plots are given in figure 12.

```
datagen1 = tf.keras.preprocessing.image.ImageDataGenerator(
                                                            rotation_range=10,
                                                            height_shift_range=0.1,
                                                            rescale=1./255,
                                                            shear_range=0.2,
                                                            zoom_range=0.1,
                                                            horizontal_flip=True,
                                                            preprocessing_function = tf.keras.applications.vgg16.preprocess_input
                                                           )

train_gen1 = datagen1.flow_from_directory(directory = TRAIN_PATH,
                                          batch_size = BATCH_SIZE,
                                          seed = 1,
                                          color_mode = 'rgb',
                                          shuffle = True,
                                          classes=classes,
                                          target_size = (IMAGE_SIZE, IMAGE_SIZE))

test_gen1 = datagen1.flow_from_directory(directory = TEST_PATH,
                                         batch_size = BATCH_SIZE,
                                         seed = 1,
                                         color_mode = 'rgb',
                                         shuffle = True,
                                         classes=classes,
                                         target_size = (IMAGE_SIZE, IMAGE_SIZE))
```

Figure 11: Data Augmentation

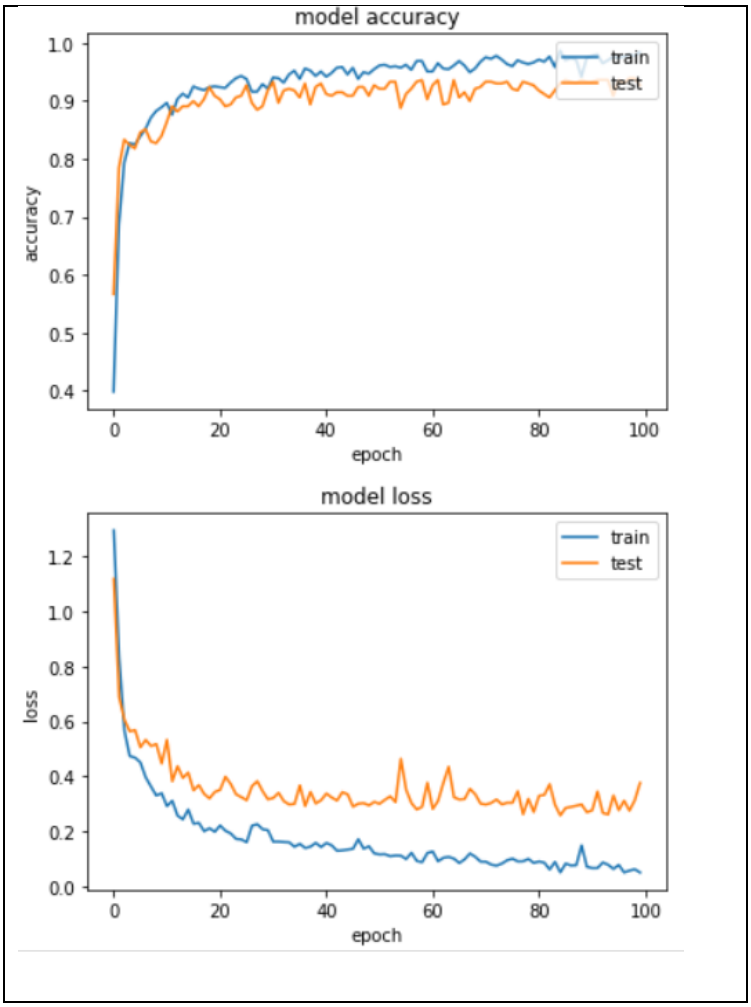


Figure 12: Model performance with augmented data

d. Model Regularization

Next, the model was modified by adding early stopping criteria and monitoring the performance after training and testing with the same augmented dataset. The relevant code snippet and the performance plots are given under Figures 13 and 14 respectively.

```
earlystopping = tf.keras.callbacks.EarlyStopping(monitor = 'val_accuracy',
                                                 min_delta = 1e-4,
                                                 patience = 5,
                                                 mode = 'max',
                                                 restore_best_weights = True,
                                                 verbose = 2)

callbacks = [earlystopping]
```

```
# Model Training with augmented dataset and early stoping
history = model.fit(
    x=train_gen1,
    steps_per_epoch=STEP_SIZE_TRAIN,
    validation_data=test_gen1,
    validation_steps=STEP_SIZE_TEST,
    epochs=EPOCHS,
    verbose=2,
    callbacks = callbacks
)
```

Figure 13: Model regularization with early stopping

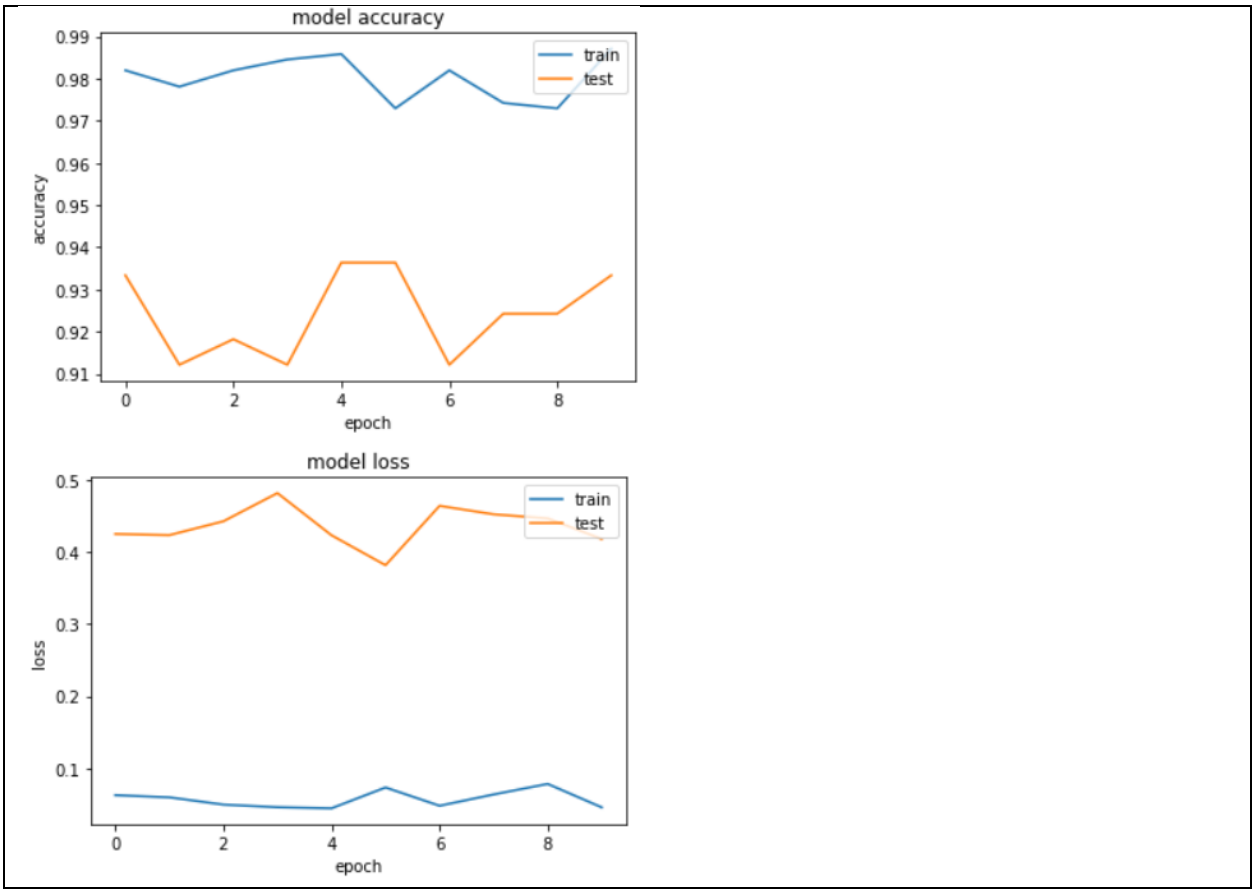


Figure 14: Model performance with early stopping

Task 4 : Results and Discussion

a. Summary of experimental results

The following table II shows the results of the experiments performed for this assignment.

- The Result1 corresponds to the training and testing of the initial model with 5 convolutional layers followed by 5 MaxPool2D layers, Flatten and Dense layer using the non-augmented dataset(results1).
- Results 2 is from the training and testing of the same model with the augmented data.
- Results 3 was recorded after updating the model with the early-stopping technique and with augmented data.
- As the results were not satisfactory, the last model(with early stopping) was modified by removing a convolutional layer with 200 neurons and adding a batch normalization layer and Result 4 was gained when it is trained with augmented data.

Table II: Overall experimental results

Result No	Accuracy		Loss		Time for training
	Training	Testing	Training	Testing	
Result 1	1.000	0.926	0.004	0.564	1 hr
Result 2	0.971	0.917	0.075	0.358	2 hrs
Result 3	0.954	0.947	0.127	0.255	11 mins
Result 4	0.796	0.788	0.489	0.598	37 mins

b. Comparision on model results

i. Compare results with Original and Augmented data

The results of the model trained with the original data were not good compared to the results of the model with augmented data. The training and testing accuracies of in the model with original data were increased gradually and become almost stable after around 30th epoch. However, the loss of the second model(with augmented data) has shown poor results as the loss was rapidly increased after around 20th epoch. This proves that there was an overfitting issue where the results were good for training, but not for testing. Further, the results of the model with augmented data were better, as the accuracies of both training and testing were slightly increasing and the loss of both training and testing was slightly decreasing. Even though the overfitting issue has almost been resolved here, still the testing accuracy is below compared to training accuracy.

ii. Compare results with attribute and model regularization

When comparing the results of the models with and without the early stopping technique, the model with early stopping was much more efficient as it could give almost similar results within a short period. The processing duration ratio between these two models was comparatively very different and that was around 11: 1. However, the accuracies of the model with early-stopping

were not significantly increasing and loss did not significantly decrease compared to the results of the model without early stopping.

c. Analysis of hyperparameter and other model settings

As the best hyperparameters for the model are not known, a trial and error approach was conducted to find them. Following hyperparameters were used to train the initial model.

- Activation function: Hidden layers: Relu, Last layer: Softmax for multiclass classification
- Number of trainable parameters: 131, 156
- Optimizer: Adam
- Learning rate: 0.0001
- Loss function: categorical_crossentrphy – for multiclass classification
- Batch size: 10
- image size: 256x256

Later, the model results were improved by changing the hyperparameters as follows.

- Number of trainable parameters: 90, 196
- Learning rate: 0.00001

Here, the last convolutional layer with 128 neurons was removed to reduce the complexity of the model and a new batch normalization layer was added before the last hidden layer to normalize the inputs to the next layer.

The summary and the results of this new model with augmented data was given below in Figures 15 and 16 respectively. This model shows a better result compared to all the other previous models and it has solved the overfitting issue as well. Both training and testing have shown an increasing accuracy and decreasing loss and the accuracy values of both training and testing were almost similar. The final result was gained by restoring to the performance of 30th epoch.

```
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d_25 (Conv2D)          (None, 256, 256, 8)       224

 max_pooling2d_25 (MaxPoolin  (None, 128, 128, 8)      0
 g2D)

 conv2d_26 (Conv2D)          (None, 128, 128, 16)      1168

 max_pooling2d_26 (MaxPoolin  (None, 64, 64, 16)       0
 g2D)

 conv2d_27 (Conv2D)          (None, 64, 64, 32)        4640

 max_pooling2d_27 (MaxPoolin  (None, 32, 32, 32)       0
 g2D)

 conv2d_28 (Conv2D)          (None, 32, 32, 64)        18496

 batch_normalization_8 (Batc  (None, 32, 32, 64)       128
 hNormalization)

 max_pooling2d_28 (MaxPoolin  (None, 16, 16, 64)       0
 g2D)

 flatten_5 (Flatten)         (None, 16384)             0

 dense_5 (Dense)             (None, 4)                 65540

=================================================================
Total params: 90,196
Trainable params: 90,132
Non-trainable params: 64
_________________________________________________________________
```

Figure 15: Summary of the new model

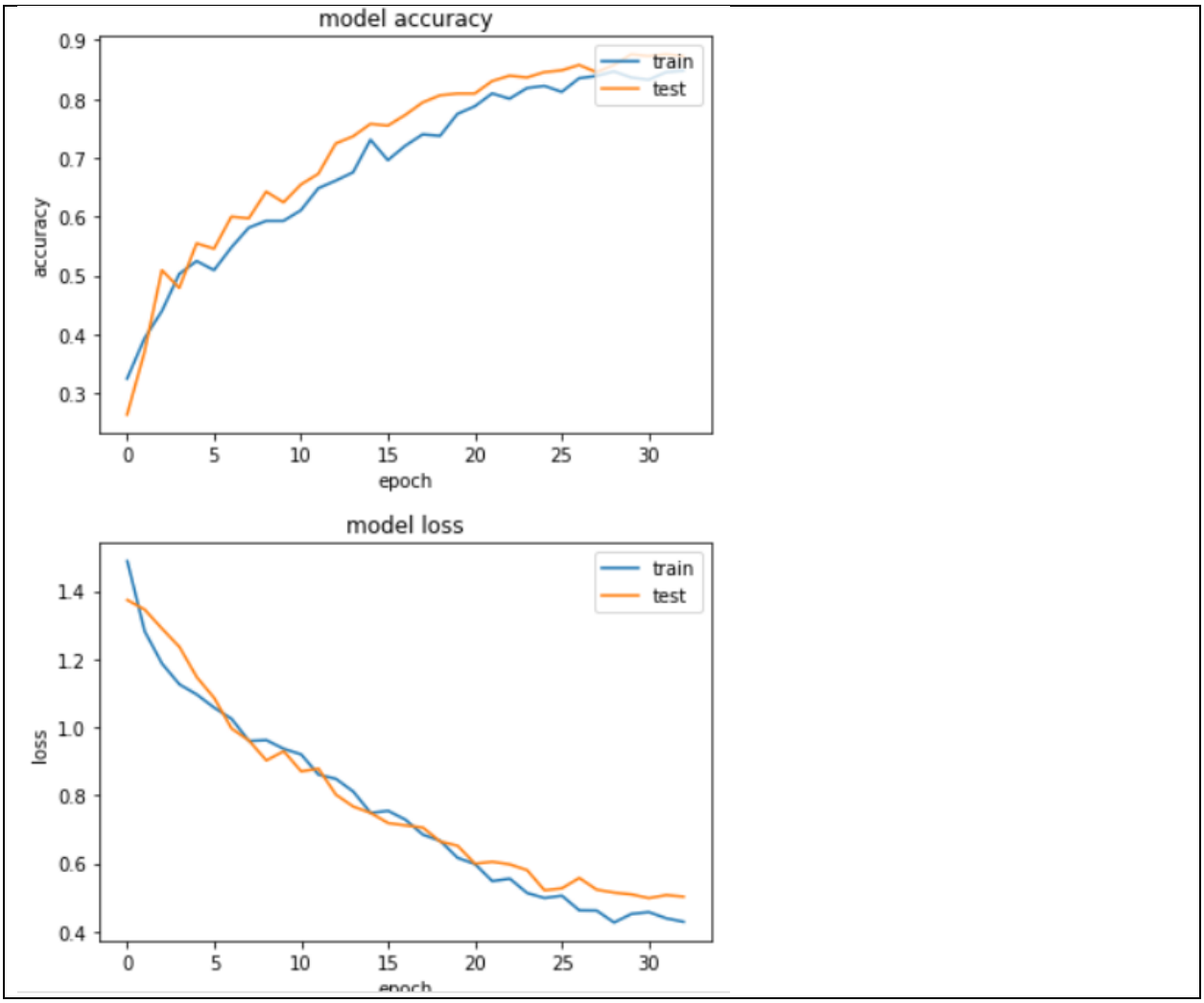


Figure 16: performance plots of the new model

d. Methodology and Implementation Strategies

Strategies on selecting the number of neurons in convolutional layers

Less number of neurons in the model can lead to underfitting and a high number of neurons can lead to overfitting. Hence, a moderate number of neurons have been used and the neurons in each successive layer will be increased in the count. The model with four convolutional layers with total of 90,132 trainable parameters has shown better results than the model with five convolutional layers that consisted of a total 131, 156 trainable parameters.

The reason for adding MaxPooling layer

These layers were used to reduce the dimensionality of features maps generated during model training. This has improved the accuracy by reducing the number of learnable parameters and further reducing the training computational time. The pool_size=(2, 2) and stride=2 are common choices.

The reason for adding batch normalization layer

As the weights of each layer change the inputs for the next layers will be changed, hence the distributions changes with each successive layer. This will cause higher training time. As it took higher time to train the model with augmented data, few batch normalization layers were added to the model.

Experimenting with adding a dropout layer

Sometimes, when the model is too complex, it will try to learn unnecessary patterns and can result in overfitting. To avoid these several dropout layers were added, so that it will disable several neurons in the model and will avoid overfitting by reducing the complexity of the model. dropout value can be any value between 1-0 and here 0.5 was added. However, as the results of this model were not good compared to the models with only other layers, hence this layer was removed from the final model.

Flatern Later

To flat a multidimensional input into one dimensional, which is coming from the convolutional layer before passing into the final dense layer.

e. Future Direction

This work can be further improved by experimenting with more data augmentation techniques. Moreover, only a few attempts were taken to test the performance with different hyperparameter settings and more configurations can be experimented to gain a better result. Another main problem was the higher computation time taken for processing the model with augmented data and without early stopping. To solve this, better resources, such as more memory and high-power processors can be used.